

Host-Based Intrusion Detection System (HIDS) Using the ELK Stack and Suricata.

**PREPARED BY:
FAWAZ ABDUL AZEEZ**

CONTENTS

INTRODUCTION	-----1
METHODOLOGY	-----3
INSTALLATION	-----4
CONFIGURATION	-----6
ELASTICSEARCH	-----6
KIBANA	-----8
LOGSTASH	-----13
SURICATA	-----16
FILEBEAT	-----18
DASHBOARD SETUP	-----23
CONCLUSION	-----26

INTRODUCTION

In this project, a Host-Based Intrusion Detection System (HIDS) was built by configuring Suricata together with the ELK Stack (Elasticsearch, Logstash, and Kibana) to capture, process, and visualise real-time network events on a Kali Linux host.

1) What is a HIDS?

A Host-Based Intrusion Detection System is a cybersecurity tool that monitors and analyses traffic to and from a host for suspicious activity, policy violations, or potential attacks in real time. When malicious activity is detected, the HIDS generates alerts for a security analyst to investigate.

2) What is the ELK Stack?

The ELK Stack is an open-source log management and data analysis platform used for collecting, storing, analysing and visualizing logs from multiple sources.

ELK stands for:

Elasticsearch: A search and analytics engine that stores and indexes data for fast searching and analytics

Logstash: A data processing pipeline that collects, filters, and forwards data to Elasticsearch.

Kibana: A visualisation tool that displays data stored in Elasticsearch using dashboards and graphs.

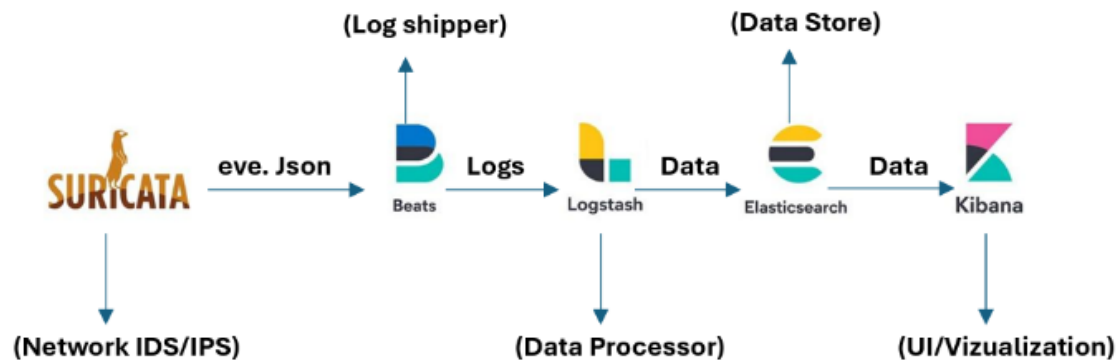
3) What is Suricata?

Suricata is an open-source Intrusion Detection System (IDS), Intrusion Prevention System (IPS), and Network Security Monitoring (NSM) engine developed by the Open Information Security Foundation (OISF).

It monitors network traffic in real time and inspects packets using rule-based signatures, logging any matches for further analysis.

Suricata generates the **eve.json** file through its **EVE (Extensible Event Format)** logging module, which is designed to output structured, machine-readable logs for use in analysis platforms like the ELK Stack.

Data flow in this project:



- Suricata writes EVE JSON logs to disk.
- Filebeat monitors these logs and sends them to Logstash.
- Logstash processes and enriches the logs.
- Elasticsearch stores and indexes the processed logs.
- Kibana queries Elasticsearch to visualize and analyse the Suricata event data.

METHODOLOGY:

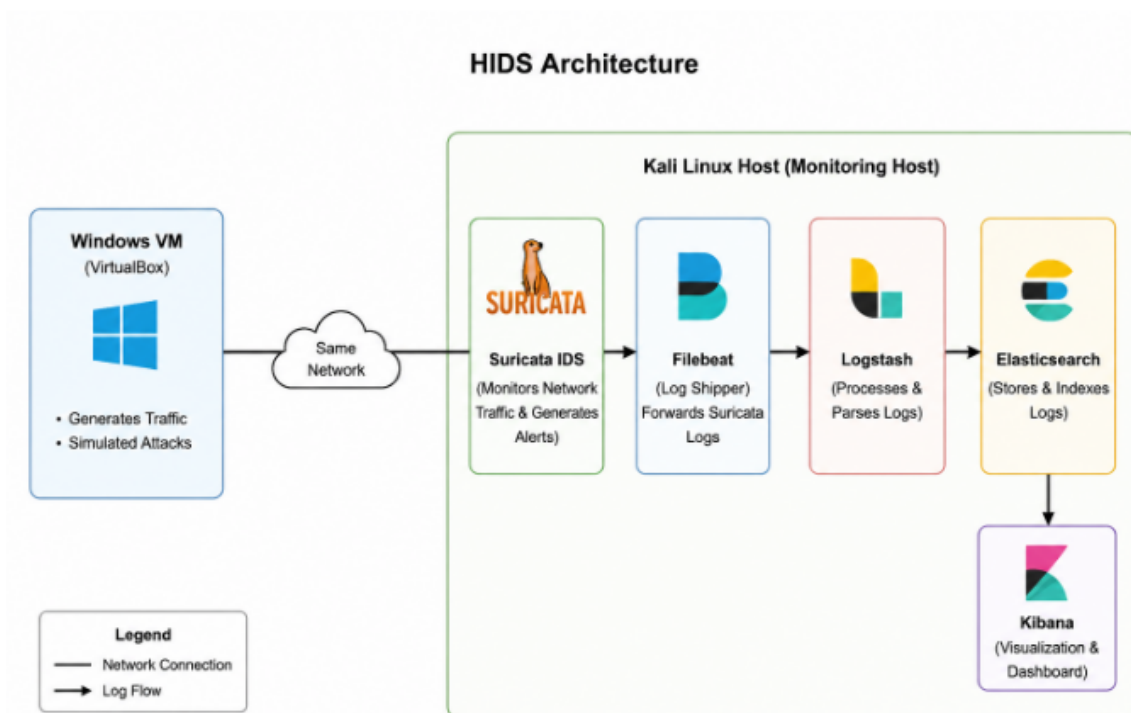
1) Tools Required:

- Elasticsearch
- Logstash
- Kibana
- Filebeat
- Suricata

2) OS Used:

The Monitoring Host (Kali Linux): This acts as the absolute central brain of the project. **All five security tools** (Suricata, Filebeat, Logstash, Elasticsearch, and Kibana) are installed and run natively here. It sniffs live traffic from your physical network interface, processes the logs, stores them, and hosts the visual dashboard console.

The Traffic Generator (Windows VM): This virtual machine runs inside VirtualBox on the same local network. It is utilized strictly as an external node to generate standard network traffic and perform simulated security attacks against the Kali



Linux monitoring host to test the detection pipeline.

INSTALLATION

1) Elasticsearch:

Download the Elasticsearch package from [elastic.co](https://www.elastic.co/downloads/elasticsearch):

<https://www.elastic.co/downloads/elasticsearch>

Here I am using Kali Linux, so I am going with the **deb x86_64** package.

Install Elasticsearch:

```
dpkg -i package_name.deb  
sudo dpkg -i elasticsearch-9.4.2-amd64.deb
```

Start the Elasticsearch service:

```
sudo systemctl enable elasticsearch  
sudo systemctl start elasticsearch
```

To check the Elasticsearch service status:

```
sudo systemctl status elasticsearch
```

2) Logstash:

Download Logstash.

<https://www.elastic.co/downloads/logstash>

Install logstash:

```
dpkg -i package_name.deb
```

Start Logstash service:

```
sudo systemctl enable logstash  
sudo systemctl start logstash  
sudo systemctl status logstash
```

3) Kibana:

Download Kibana from elastic.co:

<https://www.elastic.co/downloads/kibana>

Here I am using Kali Linux, so I am going with the **deb x86_64** package.

Install Kibana:

```
sudo dpkg -i kibana-9.4.2-amd64.deb
```

Start the Kibana service:

```
sudo systemctl enable kibana  
sudo systemctl start kibana
```

To check the Kibana service status:

```
sudo systemctl status kibana
```

4) Filebeat:

Download the Filebeat package from elastic.co using the terminal:

```
wget https://artifacts.elastic.co/downloads/beats/filebeat/filebeat-8.19.0-  
amd64.deb
```

Here I am using Kali Linux, so I am going with the **deb x86_64** package version 8.19.0.

Install Filebeat:

```
sudo dpkg -i filebeat-8.19.0-amd64.deb
```

Start the Filebeat service:

```
sudo systemctl enable filebeat  
sudo systemctl start filebeat  
sudo systemctl status filebeat
```

5) Suricata:

Add the official OISF repository to Kali Linux to fetch the stable engine build:

```
sudo add-apt-repository ppa:oisf/suricata-stable
```

Update your local package lists:

```
sudo apt update
```

Install Suricata:

```
sudo apt install suricata -y
```

Start and enable the Suricata service:

```
sudo systemctl enable suricata  
sudo systemctl start suricata
```

To check the Suricata service status:

```
sudo systemctl status suricata  
sudo systemctl status suricata.service
```

The next step is to configure these tools together by editing their .yaml, .yml, .conf, and rules files to establish the data pipeline.

Note:

Don't forget to restart the service after editing its configuration file if the services are currently running.

For example, if you edit the elasticsearch.yml file, you must restart the service using:

```
sudo systemctl restart elasticsearch
```


CONFIGURATION

1) Configure Elasticsearch:

Go to /etc/elasticsearch directory and edit the elasticsearch.yml

`sudo nano /etc/elasticsearch/elasticsearch.yml`

`network.host: "0.0.0.0"`

`#using 0.0.0.0 exposes the service to all network connections`

`http.port: 9200`

```
# ----- Network -----  
#  
# By default Elasticsearch is only accessible on localhost. Set a different  
# address here to expose this node on the network:  
#  
network.host: "0.0.0.0"  
#  
# By default Elasticsearch listens for HTTP traffic on the first free port it  
# finds starting at 9200. Set a specific HTTP port here:  
#  
http.port: 9200  
#  
# For more information, consult the network module documentation.  
#
```

Save and exit: Ctrl+X ,Y, then Enter

```
# Enable encryption and mutual authentication between cluster nodes  
xpack.security.transport.ssl:  
  enabled: true  
  verification_mode: certificate  
  keystore.path: certs/transport.p12  
  truststore.path: certs/transport.p12  
# Create a new cluster with the current node only  
# Additional nodes can still join the cluster later  
cluster.initial_master_nodes: ["Fawaz"]  
  
# Allow HTTP API connections from anywhere  
# Connections are encrypted and require user authentication  
http.host: "0.0.0.0"  
  
# Allow other nodes to join the cluster from anywhere  
# Connections are encrypted and mutually authenticated  
transport.host: "0.0.0.0"  
  
#----- END SECURITY AUTO CONFIGURATION -----
```

Enable and start the elasticsearch service, if already running then restart it.

```
sudo systemctl restart elasticsearch
```

If you get a permission error then run these commands

```
sudo chown -R elasticsearch:elasticsearch /etc/elasticsearch
sudo chmod -R 750 /etc/elasticsearch
```

If you want to reset the elastic password, then run this command

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic -
-url https://localhost:9200
```

```
(fawazzz@Fawaz)-[~]
$ sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic --url https://localhost:9200
This tool will reset the password of the [elastic] user to an autogenerated value.
The password will be printed in the console.
Please confirm that you would like to continue [y/N]y

Password for the [elastic] user successfully reset.
New value: drDqCDgLIL4T=OgjzASg
```

Note it somewhere, don't forget the password.

Check the elasticsearch is working or not:

```
curl -k -u elastic:<your_password> https://localhost:9200
```

Your Elasticsearch instance is running perfectly if you got a result like this:

```
(fawazzz@Fawaz)-[~]
$ curl -k -u elastic:drDqCDgLIL4T=OgjzASg https://localhost:9200
{
  "name" : "Fawaz",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "zhSca71DQvajEwd6zRNdug",
  "version" : {
    "number" : "9.4.2",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "c402c2b36d90eae29c0182f86bd9050fd0b746cc",
    "build_date" : "2026-05-25T22:10:36.017759931Z",
    "build_snapshot" : false,
    "lucene_version" : "10.4.0",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

2) Configure Kibana:

Go to /etc/kibana directory and run this command:

```
sudo nano kibana.yml
```

Uncomment these lines in the **System: Kibana Server** section:

```
server.port: 5601
server.host: "0.0.0.0"
```

```
# For more configuration options see the configuration guide for Kibana in
# https://www.elastic.co/guide/index.html

# ===== System: Kibana Server =====
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the `server.rewriteBasePath` setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""
```

In the **System: Elasticsearch** section, set the Elasticsearch host:

```
# ===== System: Elasticsearch =====
# The URLs of the Elasticsearch instances to use for all your queries.
elasticsearch.hosts: ["https://localhost:9200"]

# If your Elasticsearch is protected with basic authentication, these settings provide
# the username and password that the Kibana server uses to perform maintenance on the Kibana
# index at startup. Your Kibana users still need to authenticate with Elasticsearch, which
# is proxied through the Kibana server.
#elasticsearch.username: "elastic"
elasticsearch.password: "drDqCDgLIL4T=0gjzASg"
```

Since Kibana 9.x does not allow the elastic superuser account directly, use a **Service Account Token** instead of username and password.

Generate a service account token using this command: drDqCDgLIL4T=0gjzASg

```
sudo /usr/share/elasticsearch/bin/elasticsearch-service-tokens create
elastic/kibana kibana-token
```

```
# If your Elasticsearch is protected with basic authentication, these settings provide
# the username and password that the Kibana server uses to perform maintenance on the Kibana
# index at startup. Your Kibana users still need to authenticate with Elasticsearch, which
# is proxied through the Kibana server.
#elasticsearch.username: "elastic"
elasticsearch.password: "drDqCDgLiL4T=OgjzASg"

# Kibana can also authenticate to Elasticsearch via "service account tokens".
# Service account tokens are Bearer style tokens that replace the traditional username/password based configuration.
# Use this token instead of a username/password.
elasticsearch.serviceAccountToken: "AAEAAWVsYXN0aWMva2liYW5hL2tpYmFuYS10b2t1bi0yOjFmVjB0d1cyVGRDcDFTenRVZXZwcWc"
```

Add Kibana's encryption keys

Run this command to generate encryption keys:

```
sudo /usr/share/kibana/bin/kibana-encryption-keys generate
```

```
(root@Fawaz) ~ [etc/kibana]
# sudo /usr/share/kibana/bin/kibana-encryption-keys generate --force
## Kibana Encryption Key Generation Utility

The 'generate' command guides you through the process of setting encryption keys for:

xpack.encryptedSavedObjects.encryptionKey
  Used to encrypt stored objects such as dashboards and visualizations
  https://www.elastic.co/guide/en/kibana/current/xpack-security-secure-saved-objects.html#xpack-security-secure-saved-objects

xpack.reporting.encryptionKey
  Used to encrypt saved reports
  https://www.elastic.co/guide/en/kibana/current/reporting-settings-kb.html#general-reporting-settings

xpack.security.encryptionKey
  Used to encrypt session information
  https://www.elastic.co/guide/en/kibana/current/security-settings-kb.html#security-session-and-cookie-settings

Already defined settings are ignored and can be regenerated using the --force flag. Check the documentation links for instructions on how to rotate encryption keys.
Definitions should be set in the kibana.yml used configure Kibana.

Settings:
xpack.encryptedSavedObjects.encryptionKey: 13de63c8dcdec97947b9cfb0be71bc27be6b2bd472a697901e9a7838a3ae1049
xpack.reporting.encryptionKey: 5ef383d9725a36cc7ae83a0b101b2f214754fa4e376b9e0bc591084a247f036a
xpack.security.encryptionKey: a79272b73ad3ceab63e6a2d2dd316fb241791cb2ceb78dfef89cb4a2415f5661
```

Add the generated keys to kibana.yml:

```
# Time in milliseconds to wait for autocomplete suggestions from Elasticsearch.
# This value must be a whole number greater than zero. Defaults to 1000ms
# unifiedSearch.autocomplete.valueSuggestions.timeout: 1000

# Maximum number of documents loaded by each shard to generate autocomplete suggestions.
# This value must be a whole number greater than zero. Defaults to 100_000
# unifiedSearch.autocomplete.valueSuggestions.terminateAfter: 100000

# This section was automatically generated during setup.
#elasticsearch.hosts: ["https://127.0.0.1:9200"]
#elasticsearch.serviceAccountToken: AAEAAWVsYXN0aWMva2liYW5hL2tpYmFuYS10b2t1bi0yOjFmVjB0d1cyVGRDcDFTenRVZXZwcWc
#elasticsearch.ssl.certificateAuthorities: [/var/lib/kibana/ca_1781526704035.crt]
xpack.fleet.outputs: [{id: fleet-default-output, name: default, is_default: true, is_default_monitoring: true, type: elasticsearch, hosts: [https://127.0.0.1:9200], ca_trusted_fingerprint:
# --- Encryption Keys ---
xpack.encryptedSavedObjects.encryptionKey: b1a1ba3d9f2a5d4db74af0098e4c3ba6956ee18265f11417a2ed35dbffed881f
xpack.reporting.encryptionKey: eef1c8108f03112bb729dc2a43b306334edd4bfa953983a42c495c4bec561491c
xpack.security.encryptionKey: 95dfca3562c4619464e924c1ba7cef777c7db2a02e5ac3acbf6f70af6af23df27
```

Configure Elasticsearch CA Certificate in Kibana

Since Elasticsearch uses HTTPS, Kibana must verify Elasticsearch's identity using its CA certificate. Without this, Kibana cannot establish a secure connection to Elasticsearch.

Copy the http_ca.crt file to Kibana's configuration folder and rename it as ca.crt:

```
sudo cp /etc/elasticsearch/certs/http_ca.crt /etc/kibana/ca.crt
```

```
(root@Fawaz)-[~]  
# ls /etc/elasticsearch/certs/  
http_ca.crt http.p12 transport.p12
```

Add the CA certificate path to kibana.yml:

```
sudo nano /etc/kibana/kibana.yml
```

```
elasticsearch.ssl.certificateAuthorities: ["/etc/kibana/ca.crt"]  
elasticsearch.ssl.verificationMode: full
```

```
# Time in milliseconds for Elasticsearch to wait for responses from shards. Set to 0 to disable.  
#elasticsearch.shardTimeout: 30000  
  
# ===== System: Elasticsearch (Optional) =====  
# These files are used to verify the identity of Kibana to Elasticsearch and are required when  
# xpack.security.http.ssl.client_authentication in Elasticsearch is set to required.  
#elasticsearch.ssl.certificate: /path/to/your/client.crt  
#elasticsearch.ssl.key: /path/to/your/client.key  
  
# Enables you to specify a path to the PEM file for the certificate  
# authority for your Elasticsearch instance.  
elasticsearch.ssl.certificateAuthorities: [ "/etc/kibana/ca.crt" ]  
  
# To disregard the validity of SSL certificates, change this setting's value to 'none'.  
elasticsearch.ssl.verificationMode: full
```

Save and exit.

Restart Kibana:

```
sudo systemctl restart kibana
```

Why use a CA certificate?

When Kibana connects to Elasticsearch over HTTPS, it must verify that the Elasticsearch server is legitimate and not an impostor or attacker pretending to be Elasticsearch. The CA certificate acts as a trusted proof of identity — without it, Kibana would either reject the connection or connect insecurely.

Then open your browser and go to:

<http://localhost:5601>

Login with your Elasticsearch username and password.



Welcome to Elastic

You have logged out of Elastic.

Username
elastic

Password
••••••••••

Log in



elastic

Find apps, content, and more.

AI Agent

Home

Welcome home

Elasticsearch

Create search experiences with a refined set of APIs and tools.

Observability

Consolidate your logs, metrics, application traces, and system availability with purpose-built UIs.

Security

Prevent, collect, detect, and respond to threats for unified protection across your infrastructure.

Analytics

Explore, visualize, and analyze your data using a powerful suite of analytical tools and applications.

Get started by adding integrations

To start working with your data, use one of our many ingest options. Collect data from an app or service, or upload a file. If you're not ready to use your own data, play with a sample data set.

[Add integrations](#) [Try sample data](#) [Upload a file](#)

Cloud Connect

Use Elastic Cloud services like AutoOps and Elastic Inference Service for your self-managed clusters.

[Get started](#)

Management

Manage permissions

Control who has access and what tasks they can perform.

Monitor the stack

Track the real-time health and performance of your deployment.

Back up and restore

Save snapshots to a backup repository, and restore to recover index and cluster state.

Manage index lifecycle

Define lifecycle policies to automatically perform operations as an index ages.

[Dev Tools](#) [Stack Management](#)

Help us improve Elastic

We are always looking for ways to improve Elastic. Please take a moment to share your feedback with us.

[Not now](#) [Next](#)

3)Configure Logstash:

First make a pipeline configuration. This configuration file is the core instruction manual that tells Logstash which data to get (Input), how to clean it up (Filter), and where to send the final result (Output).

Go to **/etc/logstash/conf.d** directory and create the pipeline file:

`sudo nano /etc/logstash/conf.d/pipeline.conf`

Set the input, filter and output section like this:

```
GNU nano 9.0 /etc/logstash/conf.d/pipeline.conf
input {
  beats {
    port => 5044
  }
}

filter {
  # Parse incoming Suricata JSON data
  if [message] =~ /\{\}/ {
    json {
      source => "message"
    }
  }

  # --- ALIENVAULT OTX IP LOOKUP ---
  if [src_ip] {
    http {
      url => "https://otx.alienvault.com/api/v1/indicators/IPv4/%{src_ip}/general"
      verb => "GET"
      headers => {
        "X-OTX-API-KEY" => "6d462dd060bbded951b7a992cd3f0f8e26f8fde6a7c39d7fd1419086313092c1"
      }
      target_body => "alienvault_otx"
    }
  }

  # --- VIRUSTOTAL HASH LOOKUP ---
  # This looks up file hashes if Suricata captures a file transfer
  if [[fileinfo][md5]] {
    http {
      url => "https://www.virustotal.com/api/v3/files/%{[fileinfo][md5]}"
      verb => "GET"
      headers => {
        "x-apikey" => "a76e6562ce8ea039c4808012d3371e0f57da0afa23fe6acb3d6cb90abd77a263"
      }
      target_body => "virustotal"
    }
  }
}

output {
  elasticsearch {
    hosts => ["https://127.0.0.1:9200"]
    user => "elastic"
    password => "qwerty"

    # Comment this out:
    # ssl_certificate_authorities => ["/etc/elasticsearch/certs/http_ca.crt"]
  }
}
```

hosts => ["https://localhost:9200"]

password => "your elasticsearch password"

ssl_certificate_authorities => get this from /etc/elasticsearch/certs/http_ca.crt

Note: The filter section in this project includes two additional threat intelligence enrichment blocks — **AlienVault OTX** for IP reputation lookup and **VirusTotal** for file hash analysis — which automatically enrich Suricata alerts with external threat intelligence data before forwarding to Elasticsearch.

Copy the Elasticsearch CA certificate to Logstash

```
sudo mkdir -p /etc/logstash/certs
sudo cp /etc/elasticsearch/certs/http_ca.crt /etc/logstash/certs/
sudo chown logstash:logstash /etc/logstash/certs/http_ca.crt
sudo chmod 644 /etc/logstash/certs/http_ca.crt
```

Fix Logstash directory permissions:

```
sudo chown -R logstash:logstash /var/lib/logstash
sudo chown -R logstash:logstash /var/log/logstash
```

Note: Logstash cannot be run as root. Always run it as a managed systemd service.

Enable and start Logstash:

```
sudo systemctl enable logstash
sudo systemctl start logstash
sudo systemctl status logstash
```

Confirm Logstash is listening on port 5044:

```
sudo ss -tulnp | grep 5044
```



```
(root@Fawaz)-[/etc/kibana]
# sudo ss -tulnp | grep 5044
tcp        LISTEN 0      4096      *:5044      *:~      users:(("java",pid=860,fd=218))
```

4) Configure Suricata:

In this project the active interface was **wlan0** (IP address 192.168.1.37). Go to **/etc/suricata** and edit **suricata.yaml**:

```
sudo nano /etc/suricata/suricata.yaml
```

Set the interface in the af-packet section:


```
# Linux high speed capture support
af-packet:
- interface: wlan0
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
  # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
  # This is only supported for Linux kernel > 3.1
  # possible value are:
  # * cluster_flow: all packets of a given flow are sent to the same socket
  # * cluster_cpu: all packets treated in kernel by a CPU are sent to the same socket
  # * cluster_qm: all packets linked by network card to a RSS queue are sent to the same
  # socket. Requires at least Linux 3.14.
  # * cluster_ebpf: eBPF file load balancing. See doc/userguide/capture-hardware/ebpf-xdp.rst for
  # more info.
  # Recommended modes are cluster_flow on most boxes and cluster_cpu or cluster_qm on system
  # with capture card using RSS (requires cpu affinity tuning and system IRQ tuning)
  # cluster_rollover has been deprecated; if used, it'll be replaced with cluster_flow.
```

Confirm eve-log output is enabled:

```
# Extensible Event Format (nicknamed EVE) event log in JSON format
- eve-log:
  enabled: yes
  filetype: regular #regular/syslog/unix_dgram/unix_stream/redis
  filename: eve.json
  # Enable for multi-threaded eve.json output; output files are amended with
  # an identifier, e.g., eve.9.json
  #threaded: false
  # Specify the amount of buffering, in bytes, for
  # this output type. The default value 0 means "no
  # buffering".
  #buffer-size: 0
  #prefix: "@cee: " # prefix to prepend to each log entry
  # the following are valid when type: syslog above
  #identity: "suricata"
  #facility: local5
```

Create the rules file

Go to `/etc/suricata/rules` directory and create `local.rules`:

`sudo nano /etc/suricata/rules/local.rules`

Add these five rules

```
GNU nano 9.0 /etc/suricata/rules/local.rules *
alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
alert tcp any any -> any 22 (msg:"SSH Connection Attempt"; sid:1000002; rev:1;)
alert tcp any any -> any any (flags:S; msg:"Possible SYN Scan"; threshold:type both, track by_src, count 20, seconds 5; sid:1000003; rev:1;)
alert tcp any any -> any any (flags:FPU; msg:"XMAS Scan Detected"; sid:1000004; rev:1;)
alert icmp any any -> any any (itype:8; msg:"ICMP Echo Request"; sid:1000005; rev:1;)
```

Note: Every rule's sid must be unique, otherwise it won't work

Set the rules file in suricata.yaml:

```
## Configure Suricata to load Suricata-Update managed rules.
##

default-rule-path: /var/lib/suricata/rules

rule-files:
- local.rules

##
## Auxiliary configuration files.
##

classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
# threshold-file: /etc/suricata/threshold.config
```

Copy rules to the correct directory (Suricata's default-rule-path is /var/lib/suricata/rules/):

```
sudo cp /etc/suricata/rules/local.rules /var/lib/suricata/rules/
```

Run this command to check the rules are working properly:

```
(root@Fawaz)~[/etc/kibana]
# sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 8.0.5 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 16
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed. 5 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 5 signatures processed. 2 are IP-only rules, 0 are inspecting packet payload, 0 inspect application layer, 0 are decoder event only
Notice: suricata: Configuration provided was successfully loaded. Exiting.
```

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

Check the **eve.json** is being generated:

```
sudo tail -f /var/log/suricata/eve.json
```

```

root@Fawaz:~# cat /etc/kibana/_source/log/suricata/eve.json
# sudo tail -f /var/log/suricata/eve.json
{"timestamp":"2026-07-11T06:19:05.537967-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:07.585859-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:09.531282-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:11.686838-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:13.524848-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:15.564929-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:17.621356-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:19.566357-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:21.614277-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:23.557247-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:25.612629-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:27.552678-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:29.601700-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:31.546873-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:33.599258-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:35.540541-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i
d":2200121,"rev":1,"signature":"SURICATA Ethertype unknown","category":"Generic Protocol Command Decode","severity":3}}
{"timestamp":"2026-07-11T06:19:37.593093-0400","in_iface":"wlan0","event_type":"alert","pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_i

```

5) Configure Filebeat:

Go to /etc/filebeat and edit filebeat.yml:

```
sudo nano /etc/filebeat/filebeat.yml
```

Go to the filebeat.inputs section and set the paths.

Note: Filebeat 8.x does not support the old log input type — use filestream instead, otherwise the service will fail to start:

```

# ===== Filebeat inputs =====

filebeat.inputs:

# Each - is an input. Most options can be set at the input level, so
# you can use different inputs for various configurations.
# Below are the input-specific configurations.

# filestream is an input for collecting log messages from files.
- type: filestream

  # Unique ID among all inputs, an ID is required.
  id: my-filestream-id

  # Change to true to enable this input configuration.
  enabled: true

  # Paths that should be crawled and fetched. Glob based paths.
  paths:
    - /var/log/suricata/eve.json

```

Next comment out the Elasticsearch output section and uncomment the Logstash output section. Note: Filebeat will not start if more than one output is active at the same time:

```
# ----- Elasticsearch Output -----  
# output.elasticsearch:  
#   Array of hosts to connect to.  
# hosts: ["localhost:9200"]  
  
# Performance preset - one of "balanced", "throughput", "scale",  
# "latency", or "custom".  
# preset: balanced
```

Fix file permissions so Filebeat can read the log files:

```
sudo chmod 644 /var/log/suricata/eve.json  
sudo chmod 644 /var/log/auth.log
```

Check the connection between Filebeat and Logstash. Logstash must be running otherwise it won't connect:

```
sudo filebeat test output\
```

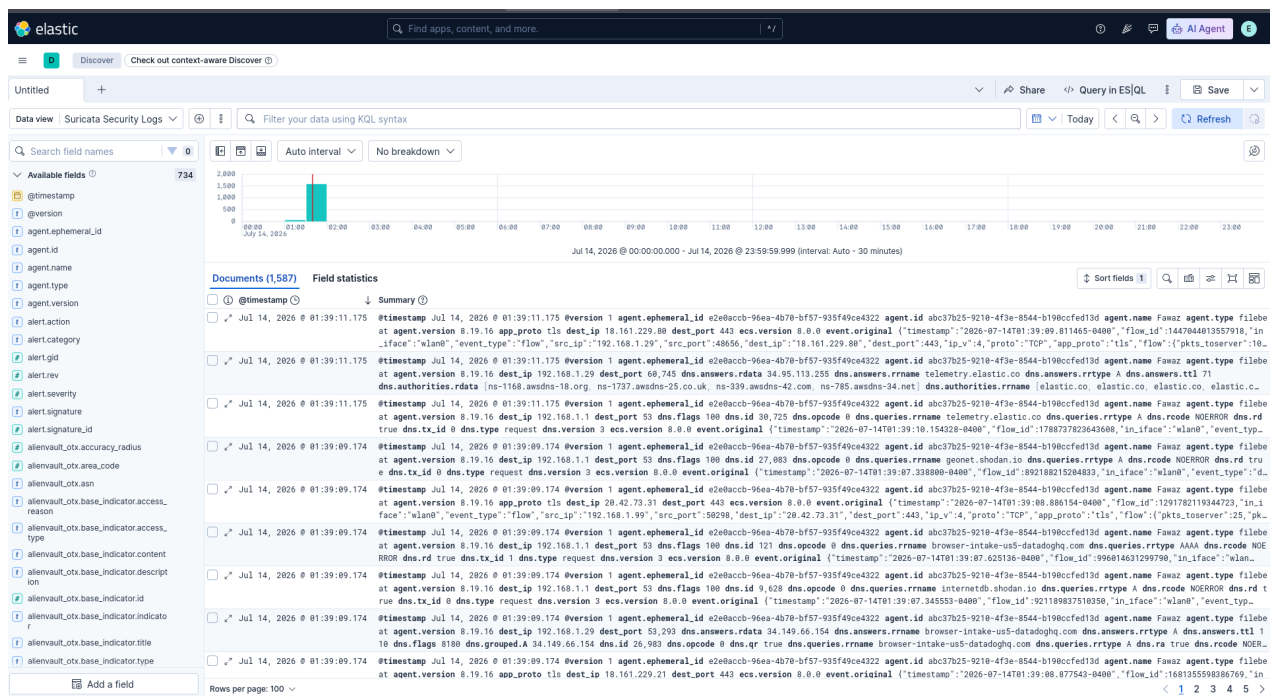
```
(root@Fawaz)-[/etc/kibana]  
# sudo filebeat test output  
logstash: 127.0.0.1:5044...  
connection...  
  parse host... OK  
  dns lookup... OK  
  addresses: 127.0.0.1  
  dial up... OK  
TLS... WARN secure connection disabled  
talk to server... OK
```

Check for Logs in Kibana

Go to Kibana → Stack Management → Data Views → Create data view:

- Index pattern: suricata-*
- Timestamp field: @timestamp
- Save the data view

Then go to Analytics → Discover. Select the Suricata Security Logs data view. You should see over 1,500 documents appearing.



Now check suricata rules are triggered. From your Windows VM, ping the Kali host

ping 192.168.1.29 -n 20

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The command entered is "C:\Users\vboxuser>ping 192.168.1.29 -n 20". The output shows 20 successful replies from 192.168.1.29, each taking less than 1ms and having a TTL of 64. Below the replies, it says "Ping statistics for 192.168.1.29:" followed by "Packets: Sent = 20, Received = 20, Lost = 0 (0% loss)," and "Approximate round trip times in milli-seconds: Minimum = 0ms, Maximum = 0ms, Average = 0ms". The prompt ends with "C:\Users\vboxuser>". The taskbar at the bottom shows various icons including the Start button, search bar, and several application icons like FIFA... and a clock showing 2:01 AM on 7/14/2026.

On Kali, check fast.log to confirm alert fired:

```
sudo tail -f /var/log/suricata/fast.log
```

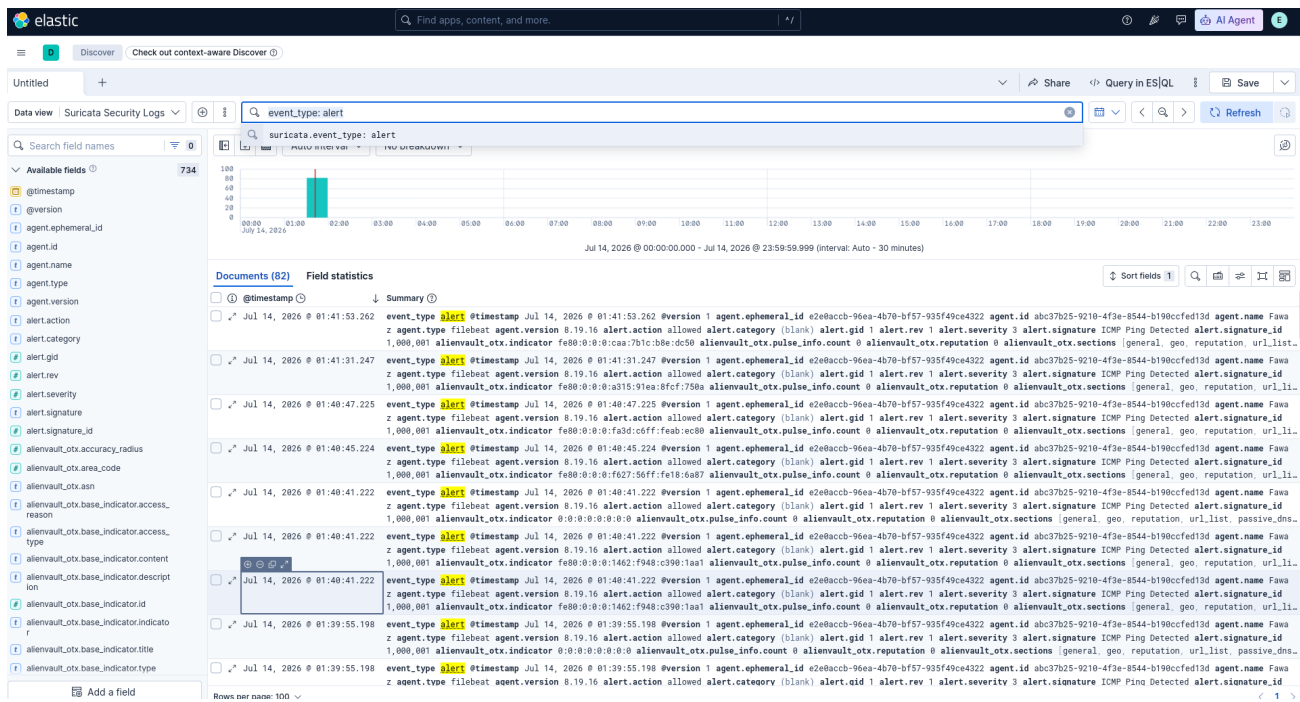
```

# root@Felix: /home/felix# ss -ttns
# sudo tail -f /var/log/suricata/fast_log

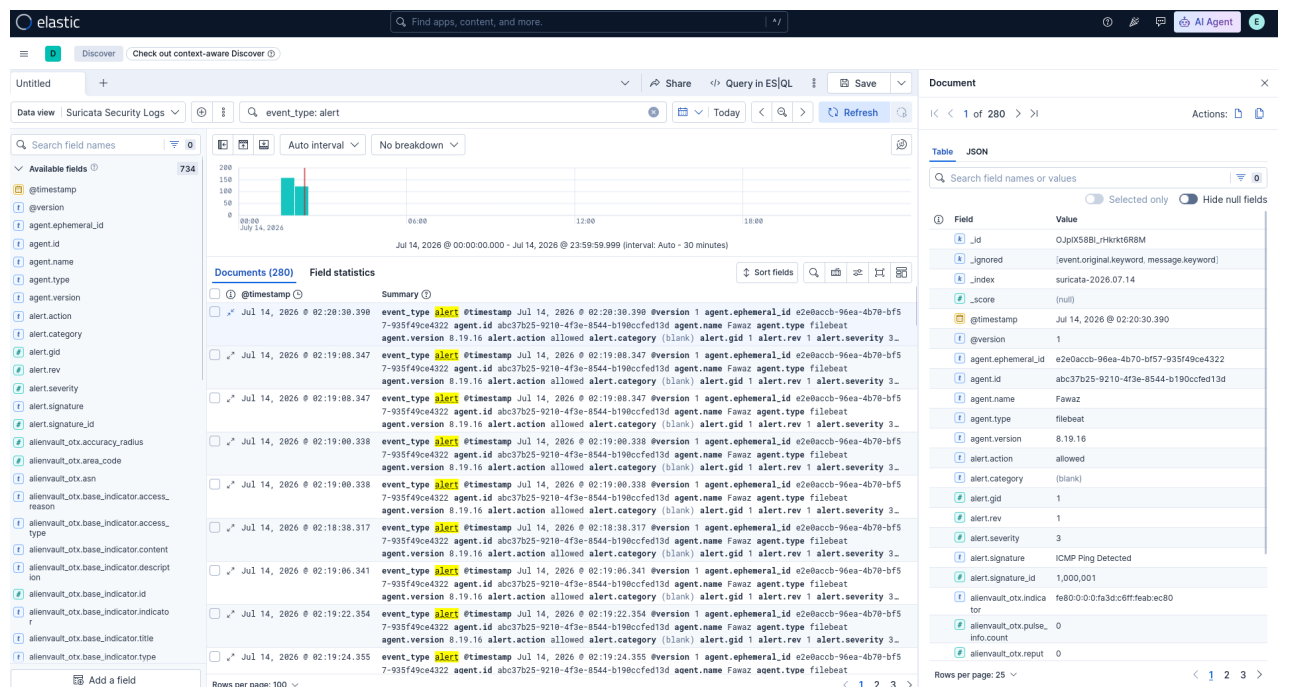
07/14/2026-01:35:38.364354 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:39.497974 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:40.428003 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:41.500656 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:42.593062 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:43.647120 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:44.691278 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:45.695063 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:46.709549 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:47.767676 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:48.794649 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:49.820297 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:50.833462 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:51.885763 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:52.937677 == [1:1:0000005:1] ICMP Echo Request == [Classification: (null)] [Priority: 3] [ICMP 192.168.1.199:8 -> 192.168.1.129:0]
07/14/2026-01:35:56.686325 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:4320:a011:f71d:2516:135 -> ff02:0000:0000:0000:0000:0001:ff55:7164:0]
07/14/2026-01:36:04.934994 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:4320:a011:f71d:2516:143 -> ff02:0000:0000:0000:0000:0000:0000:0016:0]
07/14/2026-01:36:31.564728 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:4320:a011:f71d:2516:135 -> ff02:0000:0000:0000:0000:0001:ff55:7164:0]
07/14/2026-01:36:31.564728 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:4320:a011:f71d:2516:143 -> ff02:0000:0000:0000:0000:0000:0000:0016:0]
07/14/2026-01:37:27.822235 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:1462:f948:3c90:1aa1:133 -> ff02:0000:0000:0000:0000:0000:0000:0002:0]
07/14/2026-01:37:27.822235 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:0000:0000:0000:0000:135 -> ff02:0000:0000:0000:0000:0001:ff90:1aa:1]
07/14/2026-01:37:27.913450 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:6227:56f7:fe18:b687:134 -> ff02:0000:0000:0000:0000:0000:0001:0001:0]
07/14/2026-01:37:27.913450 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:1045:209a:c22:6e95:143 -> ff02:0000:0000:0000:0000:0000:0000:0016:0]
07/14/2026-01:37:31.905852 == [1:1:000003:1] Possible SYN Scan == [Classification: (null)] [Priority: 3] [TCP 192.168.1.199:5219 -> 172.192.176.118:443]
07/14/2026-01:37:39.318087 == [1:1:000001:1] ICMP Ping Detected == [Classification: (null)] [Priority: 3] [IPv6-ICMP fe80:0000:0000:0000:4320:a011:f71d:2516:143 -> ff02:0000:0000:0000:0000:0000:0000:0016:0]
07/14/2026-01:37:40.291960 == [1:1:000003:1] Possible SYN Scan == [Classification: (null)] [Priority: 3] [TCP 192.168.1.199:5246 -> 85.210.193.152:443]
07/14/2026-01:37:40.291960 == [1:1:000003:1] Possible SYN Scan == [Classification: (null)] [Priority: 3] [TCP 192.168.1.199:5246 -> 142.250.67.35:80]
07/14/2026-01:37:44.288102 == [1:1:000003:1] Possible SYN Scan == [Classification: (null)] [Priority: 3] [TCP 192.168.1.199:5266 -> 162.159.142.9:80]

```


Now filter for alert events in kibana discover,



Expand alert document,



You can set alerts in the free Kibana version, but your options are limited to the features available with the basic license. With the free version, you can create the rules themselves like index threshold or query rules, but you are restricted to using only a limited set of actions for notifications, primarily logging to an index. Sending notifications through more advanced channels like Slack, email, or webhooks requires a paid commercial license.

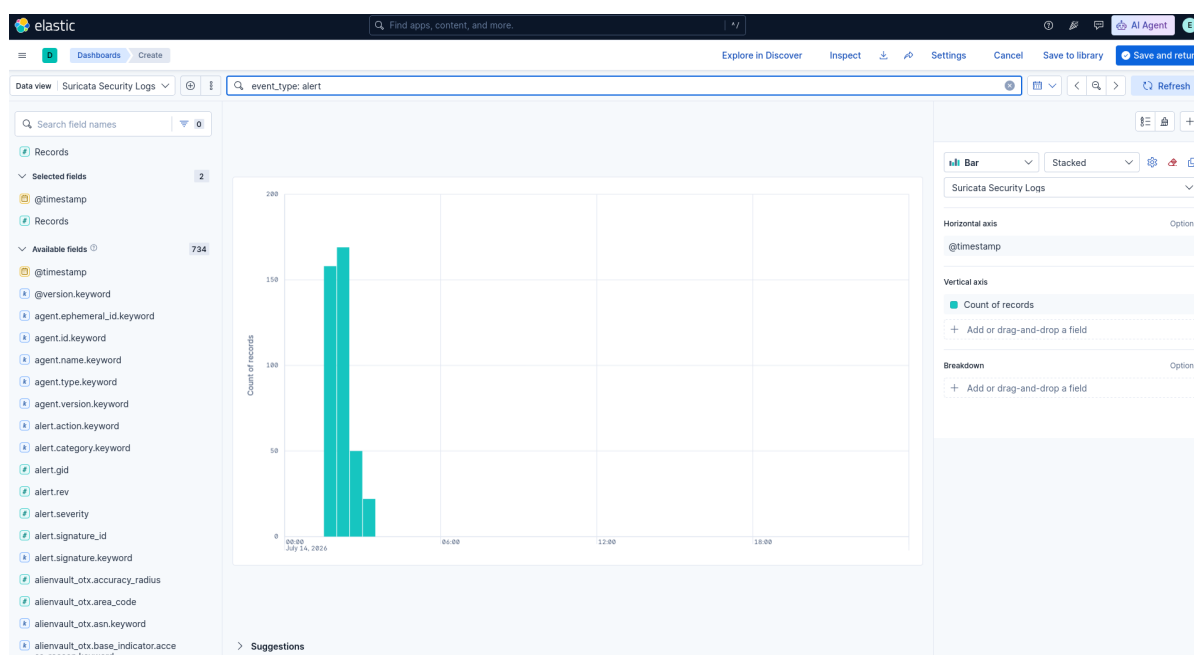
DASHBOARD SETUP:

1) Create Dashboard:

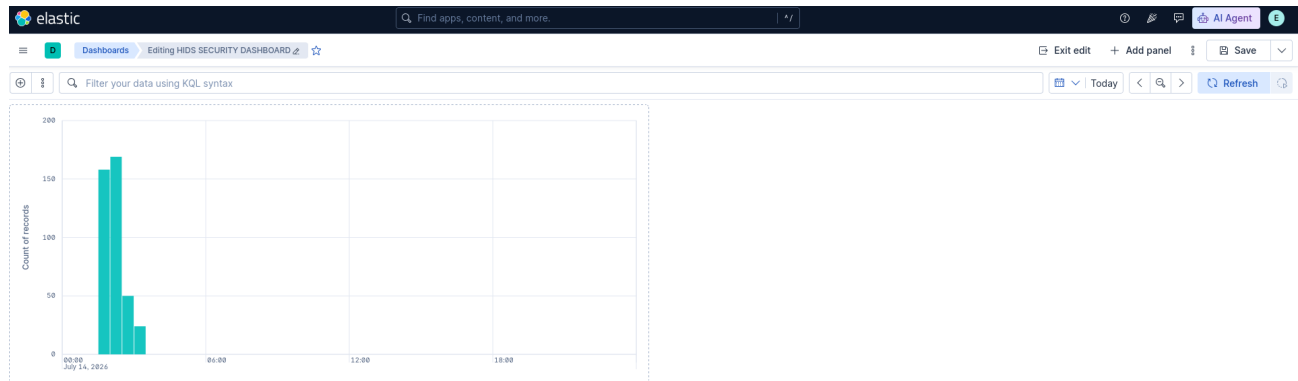
Step 1: Kibana → Analytics → Dashboards → Click **Create dashboard** → Click **Save** → Give a name → Save it.

Step 2: Analytics → Visualize Library → Create visualization → **Lens**

- Data view: Suricata Security Logs
- In the filter bar type: event_type: alert
- Set visualization type to **Bar**
- Drag @timestamp to **Horizontal axis**
- Drag **Records** to **Vertical axis**
- Click **Save and return**
- Name it **Alert Types Breakdown**
- Add to dashboard and save



The chart clearly displays spikes of alert activity corresponding to the testing periods. The highest spike showed over 150 alerts in a single time bucket on July 14th when the Windows VM (192.168.1.99) was actively pinged the Kali host (192.168.1.29).



CONCLUSION:

In this project, a Host-Based Intrusion Detection System (HIDS) was successfully built and tested using Suricata and the ELK Stack (Elasticsearch, Logstash, and Kibana) on a Kali Linux machine.

Suricata was installed and configured to monitor live network traffic on the wireless interface (wlan0). Custom detection rules were written in local.rules to detect ICMP ping attempts, SSH brute-force attacks, and Nmap scan techniques. The rules were validated using Suricata's test mode, which confirmed all rules were successfully loaded. A Windows virtual machine running inside VirtualBox was used as the attack source. When the Windows VM (192.168.1.99) pinged the Kali host (192.168.1.29), Suricata immediately detected and logged the alerts in fast.log.

Filebeat was configured to ship the Suricata eve.json logs to Logstash on port 5044. Logstash parsed and forwarded the events to Elasticsearch. The pipeline was fully verified — 1,587 documents were indexed in Elasticsearch and alert events were confirmed in Kibana Discover using the filter event_type: alert

This project also extended the standard pipeline by integrating two threat intelligence sources into the Logstash filter — **AlienVault OTX** for IP reputation lookup and **VirusTotal** for file hash analysis. These enrichments automatically add external threat context to each Suricata alert before it is stored in Elasticsearch, making the alerts more actionable for security analysts.

A Kibana dashboard was created with a bar chart visualization titled "Alert Types Breakdown" showing alert activity over time. The chart clearly showed spikes during the testing period when the Windows VM was actively generating traffic.

Several real challenges were encountered and resolved during this project, including Elasticsearch file permission errors, Kibana rejecting the elastic superuser account (requiring a service account token instead), Logstash pipeline connection issues, and Suricata rule configuration problems. Each challenge was resolved through careful troubleshooting and provided valuable hands-on experience.

While Suricata is primarily designed as a Network Intrusion Detection and Prevention System (NIDS/NIPS), this project demonstrates that when deployed on a host's own network interface it can effectively monitor host-level traffic. In a production environment, deploying Suricata on a network gateway or SPAN port would extend this same approach to protect an entire network.

This project successfully demonstrated how open-source tools can be integrated together to build a fully working real-time intrusion detection and visualization system from scratch.